

Post-Training Evaluation and Alignment Trade-offs

pig7selene

September 5, 2026

Abstract

Post-training changes how a language model responds, not merely its next-token loss. Its evaluation must therefore assess a collection of capability, instruction-following, correctness, safety, preference, style, and operational properties under declared generation and scoring protocols. This chapter develops evaluation as a multi-objective system rather than a single benchmark score. It distinguishes human, deterministic, and model-based evaluation; explains pairwise comparison, LLM-as-a-Judge, sampling-based reasoning metrics, and Reward Model validation; and analyzes distribution shift, proxy overoptimization, benchmark contamination, capability regression, catastrophic forgetting, and the alignment tax. It concludes with a regression-suite design and the contracts needed to compare post-training checkpoints reliably.

1 Introduction

Post-training is judged by behavior. SFT changes the distribution of instruction-response continuations; preference optimization changes the relative likelihood of candidate responses; PPO and GRPO change a policy through rewards obtained from new Rollouts. A lower training loss, a higher Reward Model score, or a successful update step is evidence about one interface in this process, not a complete measure of the resulting assistant.

The difficulty is structural. A response can be helpful but factually wrong, correct but noncompliant with a requested format, safe but needlessly over-refusing, preferred by a judge but excessively verbose, or strong on an average benchmark while failing a rare high-impact case. These are not merely noisy measurements of one hidden scalar. They are partially distinct objectives that can conflict under a changed prompt distribution or decoding policy. The closing task of post-training is therefore not to identify one champion metric, but to define which evidence supports each claim and which regressions are unacceptable.

This chapter closes the Post-training section. It builds on Chapter 13’s distinction between a Reward Model score and human judgment, Chapter 14’s warning about reward overoptimization, and Chapter 17’s distinction between training-time and inference-time sampling. It does not introduce a new alignment algorithm, inference-serving system, or application framework.

2 What Post-Training Evaluation Measures

Let θ denote a checkpoint, $\mathcal{D}_{\text{eval}}$ an evaluation prompt distribution, and c a complete evaluation configuration: Chat Template, system policy, decoding settings, tools if any, answer parser, and scorer. For a dimension j , define a score abstractly as

$$M_{j(\theta; \mathcal{D}_{\text{eval}}, c)} = \mathbb{E}_{x \sim \mathcal{D}_{\text{eval}}} [s_j(x, y; c)], \quad y \sim \pi_{\theta}(\cdot | x, c). \quad (1)$$

The notation in Equation 1 makes an often-hidden dependency explicit. Changing temperature, maximum length, system message, answer extraction, or evaluator can change the distribution of y and the meaning of s_j . A reported score is therefore a property of a checkpoint **under a protocol**, not an intrinsic and context-free number attached to its parameters.

The relevant object is a vector of evidence,

$$M(\theta) = (M_{\text{cap}}, M_{\text{inst}}, M_{\text{correct}}, M_{\text{reason}}, M_{\text{safe}}, M_{\text{style}}, M_{\text{oper}}). \quad (2)$$

Here M_{cap} represents general capability, M_{inst} instruction following, M_{correct} factual or task correctness, M_{reason} reasoning and coding behavior, M_{safe} harmlessness and policy adherence, M_{style} response quality and verbosity, and M_{oper} operational behavior such as response length or latency when it is relevant. The entries are not assumed to share a unit or be combined automatically.

2.1 Capability and Alignment Are Different Claims

Capability evaluation asks whether a model can solve or perform a task: answer a factual question, reason through a problem, write executable code, or use supplied evidence. It is often strongest when an answer can be checked deterministically or against a well-specified reference. **Alignment evaluation** asks whether the model behaves according to a desired policy: follows the user’s constraints, is helpful, refuses harmful assistance appropriately, preserves useful context, and avoids undesirable style or interaction patterns.

Neither category subsumes the other. An assistant can know a correct answer yet violate a length, language, or safety constraint. Conversely, a polished and compliant response can be unhelpful because it omits a crucial fact. Instruction following is especially useful to separate from general fluency: IFEval evaluates a set of verifiable natural-language constraints instead of relying only on a judge’s impression of compliance [1]. Safety and harmlessness similarly require a declared threat model and policy boundary; they are not proven by a generic helpfulness score.

Average capability also differs from worst-case behavior. Let $\mathcal{G}$ be a set of meaningful prompt groups, such as languages, domains, risk tiers, or interaction lengths. An aggregate score may be written as

$$M_{\text{avg}} = \sum_{g \in \mathcal{G}} w_g M_g, \quad M_{\text{tail}} = \min_{g \in \mathcal{G}} M_g. \quad (3)$$

M_{avg} summarizes an explicitly weighted population; M_{tail} exposes the weakest declared group. Neither is universally the right deployment criterion. A high average can conceal an unacceptable safety or accessibility failure, while a strict minimum can be unstable when groups are small. The evaluation owner must decide which groups and thresholds represent an actual requirement.

3 Evaluation Mechanisms and Their Limits

Evaluation methods differ in the evidence they produce. A deterministic verifier checks a property specified by an executable rule: unit tests, a formal proof checker, an exact answer parser, or a constrained-format instruction. It is usually the strongest option for the property it actually decides, but it cannot judge qualities absent from its rule. Human evaluation can inspect relevance, nuance, safety, factual support, and trade-offs that resist complete formalization, but it is costly and its rubric, expertise, disagreement, and sampling strategy determine what its labels mean.

Automated learned scorers occupy a third role. A Reward Model or LLM judge can evaluate many open-ended responses under a natural-language rubric. This is valuable for fast iteration and broad coverage, but it adds another model and another distribution shift to the evaluation stack. Table 1 separates the role of these mechanisms rather than treating automation as a single category.

Mechanism	Useful evidence	What it does not establish
Deterministic verifier	Reproducible pass or failure for a declared property.	Broad helpfulness, factual scope beyond the rule, or explanation quality.
Benchmark metric	Comparable performance on a fixed prompt, protocol, and scoring rule.	Generalization beyond the benchmark distribution or contamination-free capability by itself.
Human evaluation	Judgment under a stated rubric, including nuanced trade-offs.	A universal value function or full coverage at feasible cost.
Pairwise comparison	Relative preference between two candidates in context.	A stable absolute score across arbitrary prompts and response sets.
Reward Model or LLM judge	Scalable rubric-conditioned scores or rankings.	Independent ground truth; calibration and bias must be audited.

Table 1: Evaluation mechanisms make different claims. Strong evaluation uses them as complementary evidence rather than silently substituting one for another.

3.1 Pointwise, Pairwise, and Human Evaluation

Pointwise evaluation assigns one response a score or pass/fail outcome. It is efficient when a reference answer or verifier exists, but an absolute score can be sensitive to rubric wording and calibration.

Pairwise evaluation asks which of two responses better satisfies a criterion. It directly represents the relative supervision introduced in Chapter 13 and often makes subtle trade-offs easier to judge, though ranking many systems requires more comparisons and a pairing policy.

For a prompt x and two candidate systems A and B , a pairwise evaluation can be summarized by a win probability

$$W_{A,B} = P(A \text{ wins} \mid x, y_A, y_B, c), \quad y_A \sim \pi_{A(\cdot \mid x, c)}, \quad y_B \sim \pi_{B(\cdot \mid x, c)}. \quad (4)$$

The probability in Equation 4 is conditional on candidate sampling and judging protocol. It is not a context-free quality ordering. Response order should be randomized or swapped, ties should be represented when the rubric permits them, and confidence intervals should reflect prompt and judge variation. Human evaluations should additionally retain the rubric version, labeling expertise, disagreement, and any reference material available to evaluators.

3.2 LLM-as-a-Judge

LLM-as-a-Judge presents a response, or a response pair, to a judge model together with a natural-language rubric. Its advantages are substantial: it scales more easily than human annotation, has lower marginal cost, can evaluate flexible criteria, and may produce a rationale that supports auditing. It is particularly useful for exploratory evaluation and for scoring open-ended outputs that lack an exact reference.

It is not a deterministic verifier. The judge’s capability bounds the errors it can detect; a judge can be misled by an incorrect candidate even when it could solve the task independently. It can also have position bias, verbosity bias, prompt sensitivity, correlated errors with the evaluated model family, and possible self-preference. Controlled studies of LLM judges document position, verbosity, and self-enhancement concerns alongside the advantages of scalable pairwise and pointwise evaluation [2].

The appropriate response is not to discard model-based judging, but to make it auditable. Freeze and version the judge, rubric, prompt, examples, temperature, response order, and aggregation rule. Use order swaps for pairwise judgments; evaluate selected samples against independent human labels or deterministic checks; stratify by response length and task type; and report disagreement rather than forcing an apparent precision the judge does not possess. A judge score is evidence about a judge-conditioned rubric, not evidence that the target model is correct in every relevant sense.

4 Evaluation Suites, Sampling, and Regression

No benchmark suite needs to enumerate every possible task, but it must cover the failure modes its deployment claims make important. Table 2 is a practical matrix for a post-training regression suite. The goal is not to maximize every row independently; it is to prevent one strong aggregate from hiding a material loss elsewhere.

Dimension	Question and evidence	Regression guard
General capability	Can the checkpoint retain knowledge, language understanding, and transfer tasks?	Compare protected task slices with the pre-post-training baseline.
Instruction following	Does it satisfy explicit user constraints and output formats?	Use verifiable constraints and adversarially varied instructions.
Correctness and factuality	Are claims or task answers supported under a stated scorer?	Use references or checkers where available; human-audit open-ended cases.
Reasoning and coding	Can sampled solutions solve multi-step or executable tasks?	Report Pass@k, sample budget, verifier, and selected-answer metric separately.
Safety and harmlessness	Does behavior meet the declared policy across benign and adversarial prompts?	Track both unsafe compliance and inappropriate refusal on legitimate prompts.
Style and interaction	Is the response concise, relevant, and usable for the intended audience?	Control for length and evaluate preference slices with audits.
Operational behavior	Are response length, latency, tool cost, or format reliability acceptable?	Use fixed serving and decoding conditions, not an unspecified default.

Table 2: An evaluation suite should connect each claimed property to a protocol and a guard against a plausible regression. Rows may be added or removed only with a deployment-specific rationale. Sampling changes the meaning of reasoning and coding evaluation. Chapter 17 derived the idealized probability that at least one of several independent Rollouts succeeds and distinguished it from a selector’s ability to identify that Rollout. Pass@k measures coverage under a declared sampling budget; a Best-of- N or verifier-selected result also measures selection quality. Neither should be compared with a greedy Pass@1 result as though they used the same inference-time compute. Record the number of samples, decoding parameters, maximum length, answer parser, verifier version, and whether selection was oracle, learned, or rule based.

Regression testing compares the same suite across a candidate checkpoint and one or more baselines. For every dimension j , define the change

$$\Delta_j = M_{j(\theta_{\text{candidate}})} - M_{j(\theta_{\text{baseline}})}. \quad (5)$$

The vector $(\Delta_j)_j$ is more informative than an unqualified average. Release criteria can require a safety gain, permit a bounded loss on a low-risk dimension, or reject any degradation in a protected capability slice. They should also include threshold-crossing and worst-case tests, because a small global mean change can conceal a large failure in one subgroup. A regression suite is a product requirement encoded as an experiment, not merely a dashboard after training has ended.

5 Proxies, Distribution Shift, and Overoptimization

Post-training frequently optimizes a proxy: an RM score, a judge score, a benchmark score, a verifier pass rate, or a short preference rubric. Let $R(x, y)$ be such a proxy and $Q(x, y)$ the external quality property ultimately desired. A policy update that solves

$$\theta_R^* = \arg \max_{\theta}; E[R(x, y)] \quad (6)$$

need not solve the analogous objective for Q . Equality would require the proxy to preserve the relevant ordering of policy-generated responses on the distribution reached during optimization. That is a strong condition: an RM can reward polite length, an LLM judge can prefer an elaborate answer, and a benchmark can reward memorized examples. Goodhart’s Law is the practical warning that a proxy which becomes an optimization target can cease to measure the intended property reliably.

Reward overoptimization is this effect in the RM setting. Chapter 13 explained that pairwise RM accuracy does not make a scalar score a universal utility, and Chapter 14 showed why PPO constrains policy drift with reference KL. Nevertheless, optimizing strongly enough against a fixed learned

RM can produce higher R while independent human preference or task quality stalls or declines. RewardBench supplies one useful source of held-out ranking evidence for RMs, but it cannot replace evaluation of the policy that eventually searches for high scores [3]. The discrepancy must be measured on protected human, task-grounded, or adversarial evaluations.

Distribution shift compounds the issue. A Reward Model, judge, or benchmark is trained or designed around a particular set of prompts and response styles, whereas the optimized policy can generate longer, more polished, unusual, or adversarial continuations. Judge and policy errors can also be correlated when they share a model family or training data. Evaluation therefore needs a mixture of in-distribution checks, deliberately difficult contrast sets, independent judges or humans, and deterministic verifiers when a property permits them.

Benchmark contamination is a separate validity threat. If benchmark test items, answers, or strong near-duplicates enter Pretraining, SFT, preference data, or evaluator prompting, the reported performance can exaggerate generalization. The issue is not limited to exact string overlap, and for closed training corpora it can be difficult to quantify. Work on contamination argues that training on a benchmark’s test split can invalidate the intended evaluation claim and inflate performance estimates [4]. Provenance, release dates, deduplication records, contamination probes, and fresh or private holdouts are all useful controls, but none should be reported as a guarantee beyond the evidence they provide.

6 Alignment Trade-offs and Capability Regression

Alignment tax names an observed trade-off in which a post-training change improves a desired alignment objective while reducing performance on another protected capability or distribution. In the notation of Equation 5, a simple empirical pattern is $\Delta_{\text{safe}} > 0$ or $\Delta_{\text{inst}} > 0$ together with $\Delta_{\text{cap}} < 0$ on a relevant suite. This is not a theorem that alignment must reduce capability. It is a reason to report a vector of outcomes and a Pareto trade-off rather than celebrating one gain in isolation. Studies of RLHF have measured alignment-forgetting trade-offs and treat mitigation as a multi-objective problem [5].

Capability regression is the observed loss on a declared post-training comparison. **Catastrophic forgetting** is a stronger mechanism-oriented concern: adaptation on a narrower distribution can impair behavior that the earlier model could express, sometimes through a changed inferred task or prompt distribution rather than a simple deletion of all underlying knowledge. Analyses of language-model fine-tuning show why improvements on the fine-tuning distribution do not establish preservation outside it [6]. The operational response is the same: retain the pre-change checkpoint, evaluate protected capability slices, and diagnose the loss by prompt form, language, task, and sampling protocol before attributing it to one mechanism.

Safety has its own two-sided regression boundary. Lower unsafe compliance can be a genuine improvement; higher refusal on harmless, legitimate requests can be an over-refusal failure. Similarly, an instruction-tuned model can gain polished response style while losing concise directness or specialized-domain behavior. The desired result is not maximal refusal, maximal length, or maximal judge score. It is an explicitly chosen Pareto region whose thresholds reflect the deployment’s risks and users.

7 Implementation Contracts

The suite contract must version every prompt set, split, task definition, rubric, reference answer, verifier, contamination audit, and prompt-group label. It must identify which data are public, private, newly collected, or potentially exposed during Pretraining and post-training. It must retain the exact evaluation configuration c from Equation 1: system policy, Chat Template, tokenizer, decoding parameters, maximum length, tool environment, answer parser, retry policy, and scorer revision.

The judging contract must distinguish deterministic verifier, human labeler, RM, and LLM judge. For human evaluation, record the rubric, expertise requirements, candidate blinding, order randomization,

ties, disagreement, aggregation, and audit sample. For an LLM judge, record the model revision, complete judge prompt, few-shot examples, temperature, response ordering, output parser, repeated-call policy, and order-swap procedure. For a learned RM, retain the model, scalar-readout, normalization, and calibration conventions specified in Chapter 13. A judge result without this identity is not reproducible evidence.

The regression contract must name the baseline checkpoint, candidate checkpoint, suite version, statistical aggregation, uncertainty procedure, acceptance thresholds, and exception policy for every protected dimension. It should log raw score numerators and denominators where meaningful, response length and latency distributions, Pass@k sampling budget, failure examples, and both average and grouped results. A release decision should retain the report, configuration, raw outputs permitted by the data policy, and any red-team or human-audit findings. Chapter 11’s checkpoint and experiment-tracking discipline applies: an evaluation comparison is not recoverable if its model revision, data identity, or inference configuration is lost.

8 Summary

Post-training evaluation is a multi-objective comparison of behaviors under a declared generation and scoring protocol. Capability, instruction following, correctness, safety, preference, style, and operational behavior make different claims and require different evidence. Deterministic verifiers, benchmarks, human evaluation, pairwise comparison, Reward Models, and LLM judges are complementary tools with distinct validity limits.

LLM-as-a-Judge makes broad rubric-conditioned evaluation affordable, but it inherits position, length, capability, prompt, and correlation limits. Reward Models and benchmark scores are also proxies: once a model is optimized against them, their relationship to the intended quality can change. This is why independent evaluation, contamination controls, and regression suites are required rather than optional presentation polish.

The alignment tax and catastrophic forgetting are not arguments against post-training. They are reminders to compare a candidate checkpoint with a baseline across protected dimensions, groups, and failure cases. A reliable closing evaluation system records the protocol, reports the trade-offs, detects regressions before release, and reserves strong claims for the evidence its tools can actually support.

References

- [1] J. Zhou *et al.*, “Instruction-Following Evaluation for Large Language Models,” *arXiv preprint arXiv:2311.07911*, 2023, doi: 10.48550/arXiv.2311.07911.
- [2] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems 36*, Curran Associates, Inc., 2023. doi: 10.48550/arXiv.2306.05685.
- [3] N. Lambert *et al.*, “RewardBench: Evaluating Reward Models for Language Modeling,” *arXiv preprint arXiv:2403.13787*, 2024, doi: 10.48550/arXiv.2403.13787.
- [4] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. Lopez de Lacalle, and E. Agirre, “NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for Each Benchmark,” *arXiv preprint arXiv:2310.18018*, 2023, doi: 10.48550/arXiv.2310.18018.
- [5] Y. Lin *et al.*, “Mitigating the Alignment Tax of RLHF,” *arXiv preprint arXiv:2309.06256*, 2024, doi: 10.48550/arXiv.2309.06256.
- [6] S. Kotha, J. M. Springer, and A. Raghunathan, “Understanding Catastrophic Forgetting in Language Models via Implicit Inference,” in *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2309.10105.